# P2P Chat System

Phase 2 Report

Distributed Computing Project

Java Socket-Based Messaging with HDFS Directory Server

# 1. Introduction

This project builds a peer-to-peer chat system where clients can discover each other and exchange messages directly. A central directory server maintains a registry of online clients. In Phase 2, the server's storage backend is replaced with Apache Hadoop HDFS, so the client registry file is stored in the distributed file system instead of only on the local disk.

# 2. Apache Hadoop Overview

## 2.1 What is Hadoop?

Apache Hadoop is an open-source software framework for distributed storage and processing of large data sets across clusters of computers. It is designed to scale from a single machine to thousands of nodes, each providing local computation and storage. Hadoop is built around two main components:

- **HDFS (Hadoop Distributed File System):** a distributed file system that provides high-throughput access to application data, designed to run on commodity hardware.
- **MapReduce:** a programming model and processing engine for parallel computation of large data sets across the cluster.

## 2.2 HDFS

HDFS splits files into large blocks and distributes them across DataNodes in the cluster. A central NameNode keeps track of the metadata and the location of every block. This architecture provides fault tolerance through block replication and enables data locality for processing. Key HDFS shell commands used in this project include:

```
hdfs dfs -mkdir -p <path> # create a directory
hdfs dfs -put -f <src> <dst> # upload (overwrite) a local file
hdfs dfs -get -f <src> <dst> # download (overwrite) a file locally
```

## 2.3 Why Hadoop for This Project?

Storing the client registry in HDFS means the file is accessible from any node in the Hadoop cluster. If the directory server were to run on multiple machines, all instances would read and write the same shared file in HDFS, providing a single source of truth. This replaces the in-memory or purely local dictionary used in Phase 1.

# 3. System Architecture

The system has three roles:

- **Server (YServer):** a TCP server that handles client registrations, lookups, and the list of online users. It persists state in HDFS.
- **Client (YClient):** each user runs a client that registers with the server, discovers peers through the server, then sends messages directly to peers over TCP.
- **HDFS:** stores clients.txt, the flat-file database of currently online users.

Communication flow:

- Each client opens a persistent TCP connection to the server and sends an ALIVE heartbeat every second containing its user ID and listening port.
- The server writes the client record to the local file, then uploads it to HDFS.
- On every read, the server downloads the file from HDFS so the data is always from shared storage.
- When a client wants to message another user, it asks the server for that user's IP and port (GET_PEER), then opens a direct TCP connection to the peer.

# 4. Protocol

## 4.1 Client to Server Messages

```
ALIVE|<id>|<port> -> server records/updates the client

GET_LIST -> server replies with LIST|<id1,id2,...>

GET_PEER|<id> -> server replies with PEER|<id>|<ip>|<port> or
NOT_FOUND

DISCONNECT|<id> -> server removes client and replies BYE
```

## 4.2 Client to Client Messages

```
CHAT|<fromId>|<message> -> sent directly over TCP to the peer's
listening port
```

# 5. Code Structure

## 5.1 YServer.java

The server is single-entry-point: main() starts a ServerSocket on port 9000, then accepts connections in a loop. Each accepted connection is handed to a new thread running serveClient().

Key methods:

- **prepareStorage():** creates the HDFS directory and uploads an empty clients.txt on startup.
- **loadClients():** downloads clients.txt from HDFS, parses each line, and filters out entries older than 5 seconds (the timeout for dead clients).
- **saveClients():** serialises the client list to the local file and uploads it back to HDFS.
- **saveClient():** called on every ALIVE message; updates the timestamp for existing clients or inserts a new record.
- **removeClient():** deletes the client record on DISCONNECT.
- **findClient():** searches the loaded list for a specific user ID and returns the matching ClientInfo.
- **makeList():** returns a comma-separated list of all currently online user IDs.
- **runCommand():** executes an HDFS shell command using ProcessBuilder, cross-platform (Windows/Linux).

All methods that touch the client list are marked **synchronized** to prevent race conditions when multiple clients connect concurrently.

## 5.2 YClient.java

The client starts by asking for a user ID and opening a random-port ServerSocket for incoming peer messages. It then connects to the server and launches three background threads:

- **aliveThread:** sends ALIVE|<id>|<port> to the server every second to keep the registration current.
- **serverThread:** reads lines from the server and updates lastList, peerIp, peerPort, peerReady accordingly.
- **receiveThread:** accepts incoming peer connections and prints CHAT messages to the console.

The main thread runs the command menu. When the user types msg <id> <text>, sendMessage() asks the server for the peer's address, waits up to 3 seconds for a reply, then

opens a direct TCP connection to the peer and sends the CHAT message.

### 5.3 ClientInfo (inner class in YServer)

A simple data class holding id (String), ip (String), port (int), and lastSeen (long timestamp in milliseconds). Instances are created when parsing lines from clients.txt and when a new ALIVE message arrives for an unknown user.

# 6. HDFS Integration Details

Every time the server needs to read the client list it first calls getFileFromHadoop(), which runs:

```
hdfs dfs -get -f /phase2/clients.txt clients.txt
```

After every write it calls putFileInHadoop(), which runs:

```
hdfs dfs -put -f clients.txt /phase2/clients.txt
```

The **-f** flag forces overwrite so there are no errors on repeated uploads. The file format is one client per line:

```
<id>|<ip>|<port>|<lastSeen>
```

Example line:

```
alice|127.0.0.1|54321|1718000000000
```

Stale entries (lastSeen older than 5000ms) are silently dropped during the next loadClients() call, so clients that crash without sending DISCONNECT are automatically cleaned up.

# 7. How to Run

## 7.1 Prerequisites

- Java JDK 8 or later
- Apache Hadoop installed and the hdfs command available in PATH
- HDFS running (start with start-dfs.sh)

## 7.2 Compile

```
javac YServer.java
javac YClient.java
```

## 7.3 Start the Server

```
java YServer
```

## 7.4 Start Clients (one per terminal)

```
java YClient
```

Each client prompts for a user ID. After entering it the client is online. Type **list** to see who else is online, **msg <userId> <text>** to send a message, and **quit** to exit.

# 8. Conclusion

Phase 2 extends the Phase 1 chat application by replacing the in-memory or local-file client dictionary with Apache Hadoop HDFS. The server now uploads and downloads the client registry file from HDFS on every read and write operation, making the shared storage accessible to any node in a Hadoop cluster. The rest of the protocol and direct peer-to-peer messaging remain unchanged from Phase 1.